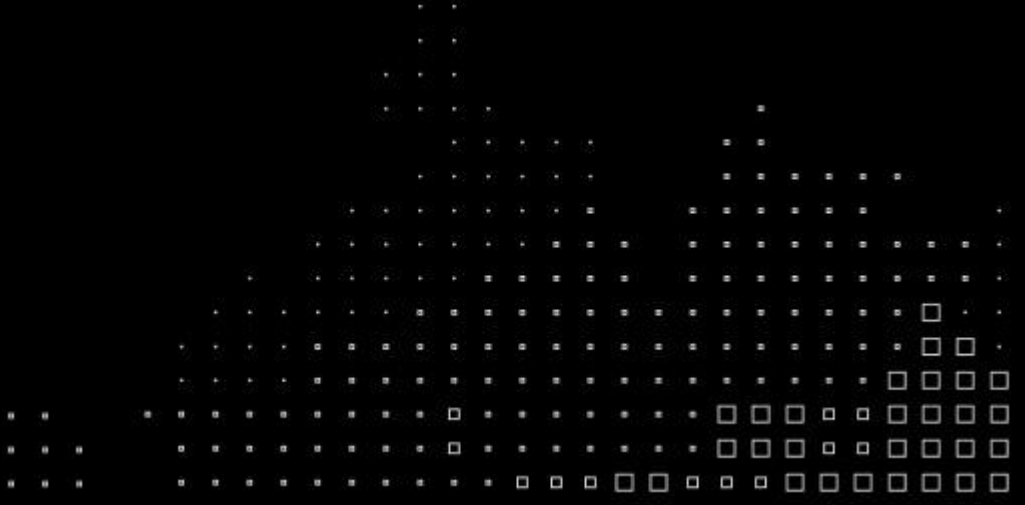
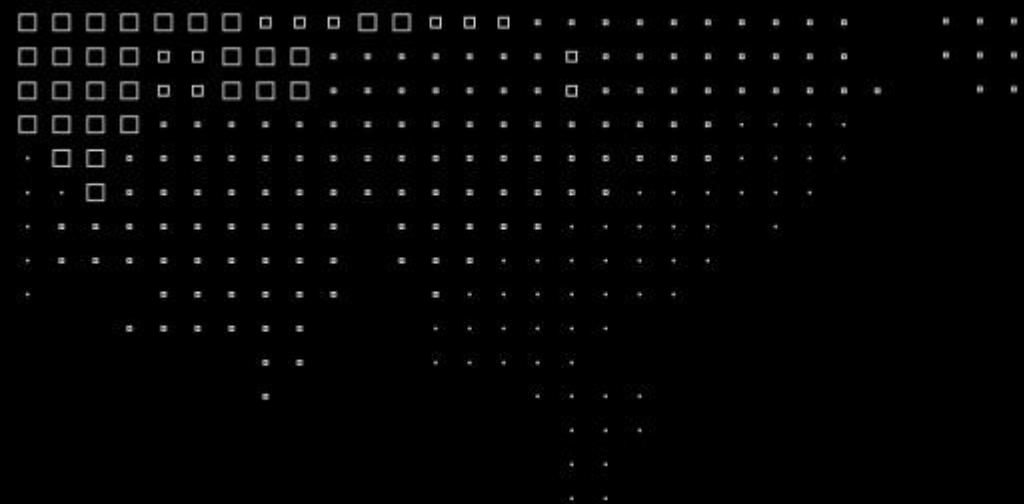


MBA⁺

AI Engineering &
Multi-Agents

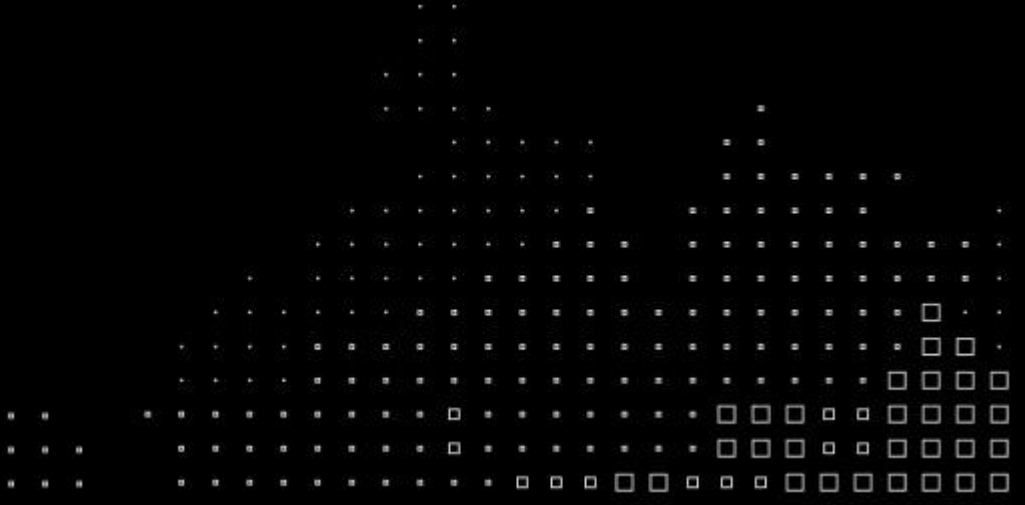




MBA⁺

Data Architecture

Prof.: Leandro Mendes



ARQUITETURA DE DADOS PARA AGENTIC AI ENGINEERING

Os Dutos do Q

Construindo a camada de dados autônoma

Ingestão, estruturação e pipelines para agentes autônomos
De uploads manuais de CSV a um ecossistema auditável e auto-orquestrado

Quando a IA vive no passado: O desafio dos dados estáticos

Três falhas reais, nenhuma delas causada pelo modelo

1

Dado estático

O agente de investimento simula rentabilidade com a taxa CDI de três meses atrás. A conta está certa. O número, não.

2

Deleção ignorada

O agente cita uma regra revogada por comunicado oficial há duas semanas. Ninguém tirou o documento do índice.

3

Eliminação incompleta

A cliente Marina pediu eliminação. O cadastro foi apagado. O agente continua lembrando o nome dela na conversa seguinte.

Diagnóstico: RAG encontra o que está indexado. Se o duto não for reativo, o modelo opera sobre uma foto antiga da empresa e defende essa foto com total confiança. O erro não está no raciocínio: está na fundação.

Por que nossos melhores modelos às vezes dão respostas ruins? O problema quase nunca é o raciocínio da IA, mas sim a fotografia desatualizada que entregamos a ela...

No laboratório: os três casos são reproduzidos hoje. A Marina existe no dataset de vocês, com id C0007, e o harness trava a nota da Missão 2 em 40 se a memória dela sobreviver ao pedido.

Paradigmas de ingestão

Quem pergunta, quem avisa, e de quanto em quanto tempo

Batch

a cada N horas, tudo de novo

- o destino pergunta
- janela fixa e previsível
- simples de operar e reprocessar
- latência = o tamanho da janela

arquivo, export noturno, dump

CDC

cada alteração vira um evento

- a origem publica
- lê o log de transações
- latência de segundos
- acoplado ao motor do banco

Debezium, log do Postgres, Change Data Feed

API

pull por polling, push por webhook

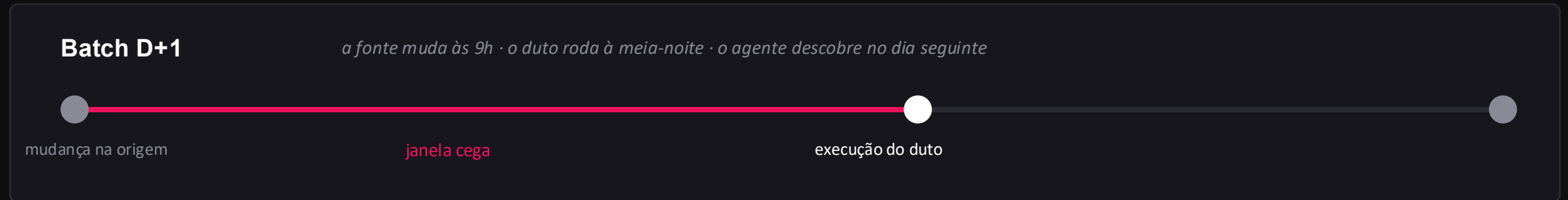
- depende de quem fala primeiro
- polling: você pergunta em intervalo
- webhook: avisam ao acontecer
- limite de taxa e reentrega

REST, GraphQL, webhook assinado

No laboratório: vocês usam os três. Batch no lote inicial de arquivos, CDC no bloco 2 lendo o Change Data Feed, e API no duto externo que traz as séries do Banco Central.

Batch: a janela de obsolescência

O intervalo em que o agente responde com confiança sobre o passado



Quando escolher batch

volume alto e tolerância a atraso;
reprocessamento simples; a origem não expõe
log nem webhook; o custo de operar streaming
não se paga

O que ele garante bem

ordem de processamento, reexecução previsível
e uma janela de manutenção onde ninguém está
lendo o meio da escrita

O que ele não garante

que o agente saiba da mudança antes da próxima
janela. Nenhum tamanho de cluster resolve: é
decisão de arquitetura, não de capacidade

No laboratório: a Missão 1 é um duto batch: 28 arquivos parados num inbox, processados duas vezes para provar que a segunda execução não muda nada.

CDC: Acompanhando a história em tempo real

A diferença entre perguntar o que existe e ser avisado do que mudou

Sem CDC

O duto relê a tabela inteira e compara com o que já tinha. Custa o tamanho da tabela a cada execução, e não sabe dizer o que foi apagado: uma linha ausente pode ter sido removida ou nunca ter existido.

Imagine ter que reler um livro inteiro apenas para descobrir se o autor adicionou uma frase nova. É esse esforço (e custo) que evitamos aqui.

Com CDC

O log de transações já registra cada linha que entrou, mudou ou saiu. O duto lê esse log a partir da última posição processada e paga o tamanho do delta, não o da tabela.

O que o feed devolve

doc_id	versao	_change_type	_commit_version	
D1	1	insert	0	
D1	1	update_preimage	2	<- como era
D1	2	update_postimage	2	<- como ficou
D2	1	delete	3	

No laboratório: no bloco 2 vocês leem esse feed cru antes de usar. Um UPDATE aparece como preimage mais postimage: é o que permite saber o que mudou, e não apenas que algo mudou.

API: polling e webhook

Duas formas de integrar com quem não te dá acesso ao banco

Polling

you ask de tempos em tempos

A favor

funciona com qualquer API, não exige endereço público, e o controle do ritmo é seu

Contra

gasta chamada mesmo quando nada mudou, e o limite de taxa define sua latência mínima

Cuidado

use janela por data e um cursor, nunca "os últimos N registros": a ordem pode mudar entre chamadas

Webhook

they notify when it happens

A favor

latência de segundos e nenhuma chamada desperdiçada

Contra

exige endpoint público, autenticação da origem e fila para absorver pico

Cuidado

entrega é at-least-once: o mesmo evento chega duas vezes, fora de ordem, e às vezes não chega. Trate com chave de idempotência e reconciliação periódica

No laboratório: o duto do Banco Central é polling com janela por data. Ele grava um snapshot na landing zone e o MERGE por (série, data) garante que rodar duas vezes não duplique uma linha sequer.

ETL e ELT

Onde a transformação acontece muda quem pode auditar o dado

	ETL	ELT
Ordem	extrair, transformar, carregar	extrair, carregar, transformar
O bruto sobrevive?	não: o destino recebe só o resultado	sim: o bruto fica na camada de pouso
Regra nova sobre dado antigo	reextrair da origem	reprocessar o que já está lá
Custo	servidor de transformação dedicado	storage barato e computação do lakehouse
Quando faz sentido	dado sensível que não pode pousar cru; origem que não aguenta extração repetida	lakehouse, dado semiestruturado, regra que ainda vai mudar
No laboratório: o kit é ELT: a Bronze guarda o arquivo como chegou e a transformação acontece depois, na Silver. É o que permite mudar uma regra do contrato e reprocessar sem pedir nada à origem.		

A Jornada do Dado: Do bruto ao refinado (Bronze, Silver e Gold)

Como organizamos e lapidamos a complexidade do mundo real

Bronze

O arquivo como chegou

Uma linha por arquivo, conteúdo bruto preservado. Nada interpretado, nada corrigido. É o que permite refazer qualquer decisão sem voltar à origem.

Bronze é o nosso porto seguro: o dado exatamente como chegou, permitindo que possamos recomeçar sem perder a história.

28 arquivos

Silver

O contrato aplicado

Tipos validados, regras de negócio, chave de idempotência, quarentena com motivo legível. Aqui o dado vira fato, ou vira rejeição explicada.

2.229 válidas · 14 em quarentena

Gold

O que o agente lê

Chunks com embedding, marcados por vigência e autoridade, mais as views com a regra de negócio embutida. A única camada que o agente enxerga.

51 chunks vigentes

No laboratório: a regra que não se negocia: o agente nunca lê da Bronze. Ali um post de blog e uma política aprovada têm o mesmo peso, e é o contrato que decide qual dos dois pode ser citado como verdade.

Metadados e contratos de dados

A system message da engenharia de dados

contratos/transacoes.yaml

```
fonte: transacoes
chave: [transacao_id]
schema_drift: quarentena
duplicatas: manter_primeira
colunas:
  cliente_id:
    fk: {tabela: clientes}
  data:
    formatos: ["%Y-%m-%d"]
regras:
  - nome: deposito_positivo
    expr: "tipo != 'deposito'
      or valor > 0"
```

Três tipos de metadado, três donos

Técnico

escreve: engenharia

tipo, nulidade, formato, chave

De negócio

escreve: o dono do dado

domínio válido, regra de sinal, vigência, autoridade da fonte

Operacional

escreve: o próprio duto

quem ingeriu, quando, quantas linhas entraram e quantas foram rejeitadas

No laboratório: o contrato de vocês vem com onze decisões marcadas como TODO. Um pipeline sem contrato não falha: ele roda, não avisa, e entrega uma tabela de aparência normal com oito linhas inválidas dentro.

Schema Drift: O que fazer quando as regras do jogo mudam

sem aviso?

A origem mudou e não avisou ninguém

ontem: transacao_id, cliente_id, tipo, valor, data

hoje: transacao_id, cliente_id, tipo, vlr, data

uma coluna renomeada. Nenhum erro, nenhum aviso, nenhum e-mail.

Aceitar em silêncio

Este é o nosso maior perigo invisível. O duto acusa sucesso, mas o valor desapareceu do saldo do cliente.

o pior dos três: falha silenciosa

Rejeitar a linha

cada linha vai para a quarentena com o motivo. O arquivo entra pela metade.

ruído: milhares de linhas idênticas na quarentena

Rejeitar o arquivo

o arquivo inteiro fica em quarentena com um motivo só. Nada entra pela metade.

escolha do kit: schema_drift: quarentena

No laboratório: o pacote do Caos traz um CSV com valor renomeado para vlr. O que o seu contrato decidir aqui vale 4 pontos no bônus, e a diferença entre uma quarentena legível e um saldo errado em silêncio.

Idempotência e Determinismo: A paz de espírito da

engenharia de dados

Rodar duas vezes, e rodar de novo daqui a seis meses

Idempotência

executar N vezes deixa o mesmo estado que executar uma vez

Determinismo

a mesma entrada produz a mesma saída, hoje e daqui a seis meses

Como se constrói

- chave natural declarada no contrato
- MERGE por essa chave, nunca APPEND
- hash da linha: só atualiza o que mudou
- registro do que já foi processado

O que quebra o determinismo

- now() e today() dentro da regra
- ordem de arquivos que muda entre execuções
- id gerado no destino em vez de vir da origem
- limite tipo "últimos 1000" sem cursor

No laboratório: vale 25 dos 100 pontos da Missão 1, o item de maior peso. O harness roda o ciclo completo duas vezes: as contagens precisam ser idênticas e a segunda execução precisa executar zero embeddings.

Time travel e auditoria de inferência

A pergunta que o regulador faz seis meses depois

“Em 14 de março o agente disse ao cliente que a tarifa era R\$ 7,00. Ele estava certo?”

Sem versionamento

you have the table today and the log of the conversation.
You can't prove what the table said that day, so the answer is “probably”.

Com time travel

each write creates a version. You read the table in the version in force on that date and compare with the answer given.

O que registrar junto

the version of the table read in each inference. Without this the time travel exists and no one knows which version to consult.

```
lake.viajar("silver.tarifas", versao=3)
```

```
lake.historico("silver.tarifas")
```

No laboratório: toda tabela do kit é Delta e guarda histórico. Depois da corrida de freshness, leiam a versão anterior de gold.chunks e vejam o índice como o agente o via antes do último evento.

O ferramental

O mesmo conceito, máquinas de tamanhos diferentes

Camada	No laboratório de hoje	Em produção
Motor de execução	delta-rs (Rust) e DuckDB, no notebook	Apache Spark ou Photon num cluster
Camada de garantia	Delta Lake: ACID, MERGE, time travel, CDF	Delta Lake, idêntico
Catálogo	views do DuckDB, criadas na sessão	Unity Catalog, com GRANT por usuário
Ingestão de arquivo	MERGE por caminho e hash do conteúdo	Auto Loader com checkpoint
Busca vetorial	produto escalar em numpy, 60 chunks	índice vetorial gerenciado
No laboratório: nada do que vocês vão decidir hoje depende do tamanho da máquina. Contrato, chave, cursor e vigência são os mesmos com 2 mil linhas e com 2 bilhões.		

Agent 1 - O Construtor

Vocês instruem, ele escreve, três filtros verificam



Por que o teste de fumaça existe

Numa versão anterior deste kit, o Construtor gerou um código que compila, passa em todos os guardrails estáticos, executa sem levantar uma única exceção, e rejeita 2.017 das 2.243 linhas: havia processado as transações antes dos clientes. Guardrail estático não pega erro de lógica.

No laboratório: o modelo é um Qwen2.5-Coder de 1,5 bilhão de parâmetros rodando em CPU no notebook. 44,7 segundos para as três lacunas, sem conta, sem token e sem cota.

Agent harness

Como se avalia um agente que produz coisa diferente a cada execução

O harness é o arcabouço em volta do agente: o que ele pode chamar, o que é bloqueado antes de rodar, e como o resultado é medido depois. Um agente sem harness é uma demo.

Superfície

quais funções o agente pode compor, e nada além disso

a Fundação lacrada: 8 funções

Guardrail

o que é reprovado sem executar

AST: import, comando destrutivo, lacuna

Eval determinística

mede o ESTADO produzido, nunca o texto gerado

contagens, quarentena, retrieval, SQL

Juíz por LLM

onde não há resposta única, um modelo julga evidências

Agent 2 sobre métricas coletadas

No laboratório: o harness de vocês avalia as tabelas, nunca o código. Um esquadrão que adotou o módulo de referência e um que gerou o próprio são medidos pelo mesmo critério, e o placar é reproduzível.

Missão 1 - O duto batch

60 minutos · esquadrões de 4 · notebook 01_bloco1_batch

O que fazer

- 1 Olhar o dado sujo antes de escrever qualquer regra
- 2 Preencher as 11 lacunas do contrato, cada uma com um custo
- 3 Escrever a system message do Construtor, que vem vazia
- 4 Revisar o código gerado e rodar o pipeline
- 5 Aos 40 minutos, sobreviver ao Caos do professor

Arquiteto decide · 2 Builders operam · Red Team lê a quarentena

Como pontua

Idempotência	25
Integridade	20
Qualidade	25
Retrieval	10
Índice limpo	10
SQL	10
Caos	-20 a +20

bronze 60 · prata 80 · ouro 95

contrato intocado = 27,5

BLOCO 2

Governança, memória e serventia

- Cursor, efeito líquido e o custo de reagir a evento
- LGPD em base vetorial e em memória de agente: o padrão outbox
- Superfície de leitura segura e menor privilégio
- Model Context Protocol: a interface entre agente e dado
- Agent 2, o Auditor, e a catraca que segura o Q

Cursor e efeito líquido

Duas decisões que separam um duto incremental de um retrabalho caro

Cursor por updated_at

Depende do relógio da origem estar certo, de ninguém gravar com timestamp antigo e de nenhuma linha chegar atrasada. Três coisas que você não controla.

Cursor por versão da tabela

Depende do log de transações, que é controlado pelo destino. “Processsei até a versão 7” é uma afirmação verificável; “processsei até as 14h32” não é.

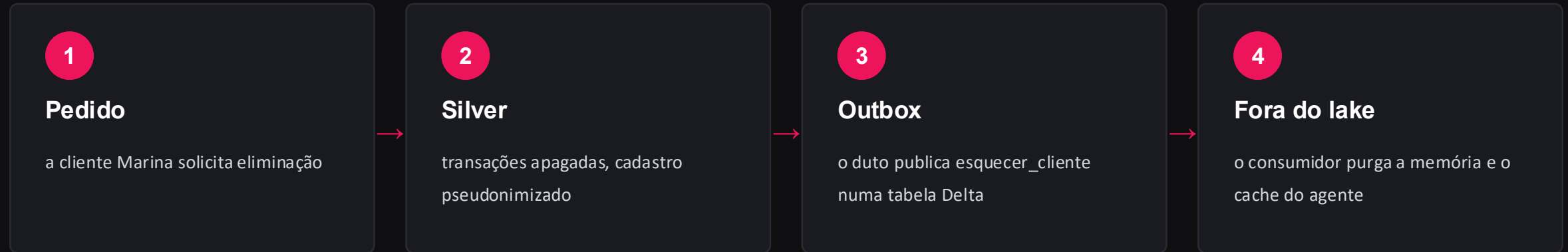
Efeito líquido: nem toda versão nova contém dado novo

Um reenvio idêntico e uma regravação completa criam versões sem que nada tenha mudado. Um duto que reage à versão re-embeda o corpus inteiro por nada. Um duto que compara o estado final da chave com o hash na Silver não faz nada, que é a resposta certa.

No laboratório: oito eventos acontecem na Missão 2 e apenas dois exigem embedding novo. Os eventos 7 e 8 existem só para punir quem reage a e vento em vez de comparar efeito. Custo vale 20 pontos.

O Direito de Ser Esquecido: Cuidando da privacidade na era da IA

Padrão outbox: a LGPD alcança quatro lugares, não um



Por que não chamar a memória direto

A memória e o cache vivem fora do lakehouse, em outro processo, que pode estar fora do ar. Uma chamada direta acopla o duto a esse processo e some com o pedido se ele falhar. O evento numa tabela sobrevive.

O ponto cego da auditoria

Apagar o cadastro do cliente do banco de dados não basta se a nossa IA continuar 'lembrando' dele através dos vetores. O padrão outbox garante que a memória do agente também respeite o pedido do cliente..

No laboratório: o evento 6 da linha do tempo é o pedido da Marina. Se a memória dela sobreviver, o harness trava a nota da missão em 40, por melhor que esteja o resto. É o artigo 18 da LGPD, não rigor de professor.

O agente invoca. Quem lê é a view.

Menor privilégio aplicado a um modelo probabilístico

Acesso direto à tabela

o que não fazer

- o modelo escreve o SQL que quiser
- joins arbitrários e consultas caras
- superfície aberta a injeção
- a regra de negócio mora no prompt

Ferramenta sobre view parametrizada

o que o kit faz

- o agente escolhe a ferramenta e passa argumentos
- consulta fixa, auditada, versionada
- sem SQL livre, sem superfície de injeção
- a regra de negócio mora na view

```
gold.v_regra_cdb → simular_cdb(valor, meses, data_aplicacao)
```

quando o comitê mudar 102% para 105%, muda a view: nenhum prompt é reescrito

No laboratório: as views do kit vivem no DuckDB, sem GRANT. Em produção a mesma view fica no catálogo com um usuário de leitura, e o UPDATE é negado pelo catálogo, não pelo código.

Model Context Protocol

O problema que ele resolve: N agentes vezes M fontes

Sem protocolo

cada agente carrega sua integração com cada fonte. Três agentes e quatro fontes são doze integrações, cada uma com sua autenticação, seu formato e seu dono.

Com MCP

cada fonte expõe um servidor, cada agente fala um protocolo. Três mais quatro em vez de doze, e trocar o agente não reescreve integração nenhuma.

As primitivas do servidor

Tools

ações que o agente invoca, com esquema de entrada declarado

`tools/list` · `tools/call`

Resources

conteúdo que o agente lê, identificado por URI

`resources/list` · `resources/read`

Prompts

modelos de interação que o servidor oferece pronto

`prompts/list`

No laboratório: hoje o kit chama funções Python diretamente. A ideia é a mesma da view parametrizada: a ferramenta é fixa e auditada, o agente só escolhe qual chamar e com quais argumentos.

Anatomia de uma ferramenta de dados

O que vocês expõem quando publicarem a Gold via MCP

tools/list devolve a descrição

```
{
  "name": "tarifa_vigente",
  "description": "Tarifa de um serviço
                 na data informada",
  "inputSchema": {
    "type": "object",
    "properties": {
      "tipo": {"enum": ["saque", "ted", "pix"]},
      "data": {"type": "string",
               "format": "date"}
    },
    "required": ["tipo", "data"]
  }
}
```

O que cada parte faz pela governança

- **description**

é o que o modelo lê para decidir se chama. Descrição vaga vira ferramenta chamada na hora errada.

- **enum no parâmetro**

o argumento inválido é barrado pelo esquema, antes de virar consulta.

- **required**

elimina a chamada pela metade, que é o erro mais comum de agente novato.

- **a implementação**

uma consulta parametrizada sobre a view. O modelo nunca vê SQL, nunca escreve SQL.

No laboratório: as ferramentas SQL do Q já têm esta forma: nome, argumentos tipados e uma consulta fixa por baixo. Publicar isso num servidor MCP é empacotamento, não redesenho.

Trabalho em Equipe: A orquestração dos nossos três Agentes

Autonomia do dado sustentando autonomia da IA

Agent 1

O Construtor

Lê o contrato e a instrução do esquadrão, escreve o código do pipeline compondo a Fundação lacrada. Não cria tabela, não escolhe nome, não escreve fora da Silver.

escreve o duto

Agent 2

O Auditor

Recebe evidências determinísticas e as regras do esquadrão, decide PASS ou FAIL e grava a catraca. Julga; não mede. Violação sem evidência real é descartada.

libera o duto

Agent 3

O Q

Chassi lacrado da aula anterior. Consome o índice e as views, e só responde quando a catraca está em PASS.

consome o duto

No laboratório: a tese da aula: um ecossistema de agentes é tão bom quanto o frescor, a qualidade e a governança da fundação de dados que o sustenta.

Missão 2 - Serventia segura e governança

60 minutos de prática · debrief no fim · notebook 02

O que fazer

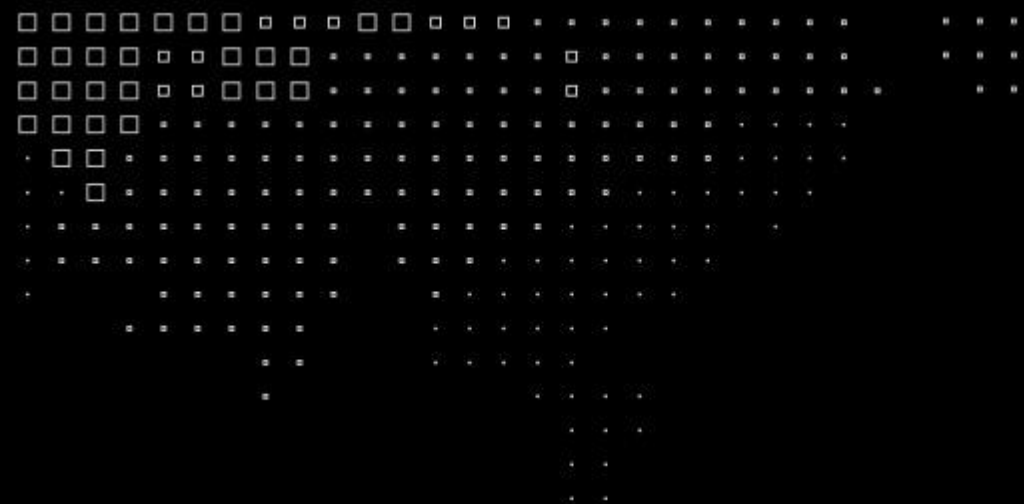
- 1 Ler o Change Data Feed cru e entender por que o cursor é a versão
- 2 Acompanhar o custo: oito eventos, quantos embeddings?
- 3 Analisar o caso Marina e o que o outbox alcança
- 4 Endurecer as regras do Auditor, que vêm frouxas de propósito
- 5 Corrida de freshness: da mudança na fonte à resposta liberada

Como pontua

Estado após cada evento	40
Deleção propagada	20
Custo de embedding	20
Freshness	20
Auditor	100 à parte

Memória da Marina viva: score travado em 40.

No laboratório: entrega: o JSON gerado pela última célula, com o score medido e as três respostas, mais o notebook salvo com as saídas.



MBA⁺

Copyright © 2026 Prof. Leandro Mendes

Todos direitos reservados. Reprodução ou divulgação total ou parcial deste documento é expressamente proibido sem o consentimento formal, por escrito, do Professor Leandro Mendes

[Material baseado em conteúdo anterior da disciplina \(FIAP\).](#)

